

1. Introduction

This project focuses on designing and implementing a graph database in Neo4j using a dataset consisting of various airlines and their arrival and departure airport details. The dataset contains information about airlines, departure airports, arrival airports, airport name, city, countries/regions, and plane types used on each route. Since the main objective is to analyse how airports are connected through airline routes, a graph database is suitable because it can naturally represent relationships between entities such as airlines and airports.

Graph Database

A graph database is a type of database that represents data using nodes, relationships, and properties. Nodes represent entities, relationships connect those entities, and properties store descriptive information about both nodes and relationships. Property graph model is where nodes are used to represent discrete objects and relationships represent the connections between them and could also contain information regarding the entities too [1].

Data Dictionary

Attribute Name	Description	Examples
Airline Name	Name of the airline	Malaysian Wings
Airline Country	Country from which the airline operates from	Malaysia
Departure Airport Name	Name of the departure airport	Ajaccio-Napoleon Bonaparte Airport
Departure Airport City	Name of the departure airport's city	Ajaccio
Departure Airport Country/Region	Name of the departure airport's country/region	France
Arrival Airport Name	Name of the departure airport	Marseille Provence Airport
Arrival Airport City	Name of the departure airport's city	Marseille
Arrival Airport Country/Region	Name of the departure airport's country/region	France
Plane Name	Name of the plane that is used for the route. Some routes use one more plane for the same route which	Airbus A320; Aerospatiale/Alenia ATR 72

	could be denoted by the ‘;’ for each of the plane.	
--	---	--

The dataset has 9 columns, and 57301 rows, which consist of 488 airlines, 2800 airports, and 114 distinct plane names.

2. Graph dataset design along with justification of decisions

2.1 Arrows App

The graph dataset was designed using the arrows app, and this is the design for this project:



Graph dataset design using arrows app.

As per the design, the Airline node stores information about each airline, which includes the attributes namely airline_name and airline_country. The reason for modelling it as a separate node was because airline related information is useful for queries such as finding all airlines from Australia in the Project. Modelling airlines as nodes is appropriate as airlines are distinct entities in the route network. Nodes are entities or discrete objects in a graph model, while relationships represent the connections between them [1].

The SERVES_FROM relationship connects an airline to the first airport node (departure airport) it operates from. This follows the property graph model, where relationships are used to represent connections between nodes. In this case, the relationship connects the airline entity to the airport entity and keeps airline information connected to the route network without duplicating airline details across multiple airport nodes [2].

The Airport node stores airport related attributes, namely airport_name, airport_city, and airport_country_region. The same Airport node type is used for both the departure and arrival sides of a route. This is because the airport is the same real world entity whether it is used as a departure airport or an arrival airport. This is also consistent with the hub and

spoke model, where a central hub connects to multiple spoke locations. In aviation, airports can act as hubs that connect passengers to other destinations [3]. For example, a passenger may travel from India to Perth through an intermediate hub airport such as Singapore or Kuala Lumpur, rather than through a direct route.

Hence, creating separate nodes for `Departure_Airport` and `Arrival_Airport` would create unnecessary data redundancy and make it complicated. Hence, the airport details such as name, city, country/region for arrival and departure were combined, and the duplicates would be dropped, thus remaining the same across departure and arrival.

The `ROUTE` relationship connects one `Airport` node to another `Airport` node. It stores the attributes of `airline_name` and `plane_type`. This is useful as routes are directional, and they describe a specific route. For example, India to Singapore and Singapore to India are separate records as per the dataset. This design allows each direction to be represented independently, which matches the project requirement that one row equals one route record.

Pros of the graph dataset design:

- The design directly supports route based queries because airports are connected through directed `ROUTE` relationships.
- Using one `Airport` node type avoids unnecessary duplication between departure and arrival airport entities.
- Storing route information on the `ROUTE` relationship is consistent with the graph database model.
- The graph database design is simple and easy to interpret, while still supporting the required project queries.

Cons of the graph dataset design:

- `plane_type` is stored as a relationship property rather than as a separate node, so aircraft specific analysis requires property filtering instead of graph traversal.
- `airline_name` is stored both in the `Airline` node and in the `ROUTE` relationship, which introduces some redundancy. However, this makes route based queries easier because each route record directly stores the airline operating it.

Justification of the graph dataset design:

This design was finalized as it was sufficient to answer all the necessary queries for the project. The current design is simple and contains two node types, `Airline` and `Airport`, and two relationship types, `SERVES_FROM` and `ROUTE`. One of the airports is the central node

with ROUTE being the relationship between two airports. And SERVES_FROM being the other relationship between airport and airline.

3. Data Cleaning, Pre-Processing, and ETL Process

3.1 Data Cleaning

Overview:

Data cleaning was performed to ensure the data quality and consistency before loading the dataset into Neo4j for creating the required nodes and relationship for the graph database. The Project2_dataset.csv file was first cleaned in Google Colab. This was done by updating the country names that were mismatched with their respective airport and country names. Then, as per the arrows app design of the graph database, the required nodes namely airport and airline csv files were created. Followed by their respective relationship csv files serves_from and route were also created before carrying on with the graph database creation.

Data Cleaning Steps

1. Reading the data:

- The CSV file was loaded into Google Colab using `pd.read_csv()`. And the pandas library was used for the data cleaning steps.

```
df = pd.read_csv('Project2_dataset.csv')
```

2. Mismatched airport city and names along with airport country/region:

- There were some dissimilarities noted between the arrival and departure airport country/region. Hence, the attributes namely, arrival airport name, arrival airport, arrival airport country/region, departure airport name, departure airport, and departure airport country/region were combined and renamed as `airport_name`, `airport_city` and `airport_country_region` respectively.
- Then, we identified airport names and city combinations linked to more than one country/region.
- After which, we filtered out only airport name and city combinations that have more than one country/region. This would be the corrections that we perform on the dataset.

```
# Departure airport attributes.
departure_airports = df[['Departure Airport Name',
                        'Departure Airport City',
                        'Departure Airport
Country/Region']].copy()
```

```

departure_airports.columns = ['airport_name', 'airport_city',
                              'airport_country_region']

# Arrival airport attributes.
arrival_airports = df[['Arrival Airport Name',
                      'Arrival Airport City',
                      'Arrival Airport Country/Region']].copy()
arrival_airports.columns = ['airport_name', 'airport_city',
                           'airport_country_region']

#Combine departure and arrival airport records to identify airport
and country mismatches.
airport = pd.concat([departure_airports, arrival_airports],
                    ignore_index=True)

#Identify airport name and city combinations linked to more than one
country/region.
airport_mismatches = (
    airport.groupby(["airport_name",
                    "airport_city"])["airport_country_region"]
    .agg(
        countries_found=lambda x: sorted(x.dropna().unique()),
        country_count=lambda x: x.dropna().nunique(),
        total_records="count"
    )
    .reset_index()
)

#Keep only airport name and city combinations that have more than
one country/region.
airport_mismatches =
airport_mismatches[airport_mismatches["country_count"] > 1]

airport_mismatches

```

3. Standardising the data:

- The data cleaning would emphasise on only on the arrival airport country/region attribute as the departure airport country/region attribute seemed already cleaned.
- Since the detected airport name to country name mismatches were found in the arrival airport country field, the correction logic was applied to the arrival airport country/region attribute. The correct values remain unchanged, while incorrect values are corrected by identifying them individually.

- The arrival airport country/region had a lot of mismatched countries with the city and airport names, and they were corrected while referencing the Airline Dataset Updated -v2.csv from Project – 1, and Wikipedia [4].

```
#Correction mapping for airport name and city combinations with the
correct airport country/region.
corrections = {
    ('Albany Airport', 'Albany'): 'Australia',
    ('Alberto Carnevali Airport', 'Merida'): 'Venezuela',
    ('Alexandria International Airport', 'Alexandria'): 'United
States',
    ('Arturo Michelena International Airport', 'Valencia'):
'Venezuela',
    ('Atlas Brasil Cantanhede Airport', 'Boa Vista'): 'Brazil',
    ('Birmingham-Shuttlesworth International Airport',
'Birmingham'): 'United States',
    ('Charles M. Schulz Sonoma County Airport', 'Santa Rosa'):
'United States',
    ('Cheddi Jagan International Airport', 'Georgetown'): 'Guyana',
    ('Cibao International Airport', 'Santiago'): 'Dominican
Republic',
    ('Cochin International Airport', 'Kochi'): 'India',
    ('Comodoro Arturo Merino Benitez International Airport',
'Santiago'): 'Chile',
    ('El Alto International Airport', 'La Paz'): 'Bolivia',
    ('Eugene F. Correia International Airport', 'Georgetown'):
'Guyana',
    ('F. D. Roosevelt Airport', 'Oranjestad'): 'Netherlands
Antilles',
    ('Florence Regional Airport', 'Florence'): 'United States',
    ('Fort Smith Regional Airport', 'Fort Smith'): 'United States',
    ('Futuna Airport', 'Futuna Island'): 'Vanuatu',
    ('General Jose Antonio Anzoategui International Airport',
'Barcelona'): 'Venezuela',
    ('JAGS McCartney International Airport', 'Cockburn Town'):
'Turks and Caicos Islands',
    ('London City Airport', 'London'): 'United Kingdom',
    ('London Gatwick Airport', 'London'): 'United Kingdom',
    ('London Heathrow Airport', 'London'): 'United Kingdom',
    ('London Luton Airport', 'London'): 'United Kingdom',
    ('London Stansted Airport', 'London'): 'United Kingdom',
    ('Mayor Buenaventura Vivas International Airport', 'Santo
Domingo'): 'Venezuela',
```

```

    ('Norman Manley International Airport', 'Kingston'): 'Jamaica',
    ('Norman Y. Mineta San Jose International Airport', 'San Jose'):
'United States',
    ('Northwest Florida Beaches International Airport', 'Panama
City'): 'United States',
    ('Presidente Joao Batista Figueiredo Airport', 'Sinop'):
'Brazil',
    ('Richmond Airport', 'Richmond'): 'Australia',
    ('San Jose Airport', 'San Jose'): 'Philippines',
    ('Santa Fe Municipal Airport', 'Santa Fe'): 'United States',
    ('Santa Rosa International Airport', 'Santa Rosa'): 'Ecuador',
    ('St Petersburg Clearwater International Airport', 'St.
Petersburg'): 'United States',
    ('St Pierre Airport', 'St. -pierre'): 'Saint Pierre and
Miquelon',
    ('Sydney Kingsford Smith International Airport', 'Sydney'):
'Australia',
    ('Tri-Cities Regional TN/VA Airport', 'Bristol'): 'United
States',
    ('Victoria Regional Airport', 'Victoria'): 'United States',
    ('Waterloo Regional Airport', 'Waterloo'): 'Canada',
}

```

4. Data with accent:

- Airport such as Comodoro Arturo Merino Benitez International Airport, General Jose Antonio Anzoategui International Airport, and Presidente Joao Batista Figueiredo Airport had accent in the data, and it was manually verified with while referencing the Airline Dataset Updated -v2.csv.

5. Ambiguous data:

- There were a few ambiguous arrival airport country/region such as for the airport Santa Ana Airport where in the Project2_dataset.csv the country was the United States. Whereas in the Airline Dataset Updated -v2.csv the arrival airport country/region was mentioned either Solomon Islands or Colombia.
- Hence, these types of ambiguous cases were not updated. And there are similar cases like these that were encountered and not updated as there could genuinely be two countries that have the same airport name.

6. Data Cleaning:

- A function to correct mismatched arrival airport country/region values. It checks whether the arrival airport name and city exist in the correction mapping. It keeps the original data, if not part of the issue. If not, it applies the corrections to the Arrival Airport Country/Region column. The cleaned data is then saved in a new csv.

```
#Function to correct mismatched arrival airport country/region values.
def country_mismatch(row):
    #Check whether the arrival airport name and city exist in the
    correction mapping.
    key = (row['Arrival Airport Name'], row['Arrival Airport City'])
    #Keep the original data, if not part the of issues.
    return corrections.get(key, row['Arrival Airport Country/Region'])

#Apply the corrections to the Arrival Airport Country/Region column.
df['Arrival Airport Country/Region'] = df.apply(country_mismatch,
axis=1)

## Save the cleaned dataset.
df.to_csv('Project2_dataset_updated.csv', index=False)
print("'Project2_dataset_updated.csv' has been created.")
```

3.2 Data Preprocessing:

1. Node files:

- The attributes `airline_name`, and `airline_country` were the only ones in the dataset for airline, and the duplicates were dropped before creating the csv file.

```
#Create the Airline node CSV by keeping distinct airline name and
country combinations.

airline = df[['Airline Name', 'Airline Country']].drop_duplicates()

airline.columns = [
    'airline_name',
    'airline_country'
]

airline.to_csv('airline.csv', index=False)
print("'airline.csv' has been created.")
```

- The node for airport was created by merging arrival airport name, arrival airport, arrival airport country/region, departure airport name, departure airport, and departure airport country/region and renaming them as `airport_name`, `airport_city` and `airport_country_region` respectively.
- Then the duplicates were dropped from the dataset before converting it to a csv file.

- In the arrows app design of this graph database, it could be noted that there are 2 airport nodes, and in this case the same node airport would be used in both the places.

```
#Recreate airport records after cleaning the arrival airport
country/region values.
departure_airports = df[['Departure Airport Name',
                        'Departure Airport City',
                        'Departure Airport Country/Region']].copy()

departure_airports.columns = [
    'airport_name',
    'airport_city',
    'airport_country_region'
]

arrival_airports = df[['Arrival Airport Name',
                      'Arrival Airport City',
                      'Arrival Airport Country/Region']].copy()

arrival_airports.columns = [
    'airport_name',
    'airport_city',
    'airport_country_region'
]

airport = pd.concat([departure_airports, arrival_airports],
                    ignore_index=True)

#Create the Airport node CSV by combining departure and arrival
airports and removing exact duplicates.
airport = airport.drop_duplicates(subset=['airport_name',
    'airport_city', 'airport_country_region']).reset_index(drop=True)
airport.to_csv('airport.csv', index=False)
print("'airport.csv' has been created.")
```

2. Relationship files:

- The SERVES_FROM relationship was created by matching each Airline node using airline_name to the departure Airport nodes it serves from using airport_name, airport_city, and airport_country_region. It was matched with all three attributes rather than using only airport_name as there are some ambiguous data where in certain airport_names had the same names across different cities or countries.

```
#Create the SERVES_FROM relationship CSV between Airline and departure Airport.
```

```
serves_from = df[[
    'Airline Name',
    'Departure Airport Name',
    'Departure Airport City',
    'Departure Airport Country/Region'
]].drop_duplicates()

serves_from.columns = [
    'airline_name',
    'airport_name',
    'airport_city',
    'airport_country_region'
]

serves_from.to_csv('serves_from.csv', index=False)
print("serves_from.csv has been created.")
```

- The ROUTE relationship was created by matching a departure Airport node to an arrival Airport node. Both endpoints were matched using the same set of attributes namely: airport_name, airport_city, and airport_country_region. It was matched with all the attributes rather than using only airport_name as there are some ambiguous data where in certain airport_names had the same names across different cities or countries.
- The Route relationship stores attributes such as plane_type and airline_name. As the plane_type is specific for each route it is stored in the relationship. The airline_name attribute is also stored on the relationship even though it is already represented in the Airline node, because it makes route related queries easier, such as identifying which airline operates a particular route, etc.

```
# ROUTE relationship exists between two airport nodes.
route = df[[
    'Departure Airport Name',
    'Departure Airport City',
    'Departure Airport Country/Region',
    'Arrival Airport Name',
    'Arrival Airport City',
    'Arrival Airport Country/Region',
    'Plane Name',
    'Airline Name'
]].copy()

route.columns = [
    'departure_airport_name',
    'departure_airport_city',
    'departure_airport_country_region',
    'arrival_airport_name',
    'arrival_airport_city',
    'arrival_airport_country_region',
    'plane_type',
    'airline_name'
]

route.to_csv('route.csv', index=False)
print("route.csv has been created.")
```

Tools that were used:

- Google Colab
- Pandas

3.3 ETL Process

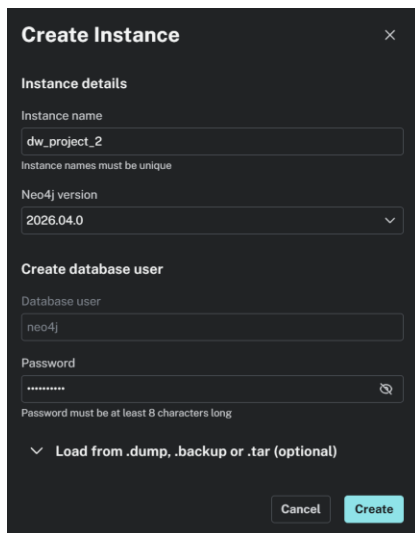
Creating an instance in Neo4j

After all the tables were created, they were all downloaded in their csv file version from Google Colab using this code:

```
from google.colab import files

files.download("airline.csv")
files.download("airport.csv")
files.download("serves_from.csv")
files.download("route.csv")
```

Then an instance by the name dw_project_2 was created using Neo4j for the project as seen in the screenshot below:

A screenshot of the 'Create Instance' dialog box in Neo4j. The dialog has a title bar with a close button (X). It contains two main sections: 'Instance details' and 'Create database user'. In the 'Instance details' section, there is a text input for 'Instance name' with the value 'dw_project_2' and a note 'Instance names must be unique'. Below that is a dropdown for 'Neo4j version' set to '2026.04.0'. The 'Create database user' section has a text input for 'Database user' with the value 'neo4j', a password input field with a toggle for visibility, and a note 'Password must be at least 8 characters long'. At the bottom, there is a checkbox for 'Load from .dump, .backup or .tar (optional)' which is currently unchecked. At the very bottom are 'Cancel' and 'Create' buttons.

Now the created instance is then connected as seen below:

✕

!

Connecting using saved credentials was unsuccessful. Connect using the correct credentials below.

Connection URL

neo4j://127.0.0.1:7687

Database credentials

Protocol

neo4j://

▼

Connection URL

127.0.0.1:7687

Database user

neo4j

Password

🗕

Connect

After which, the datasets were added to the import folder of the instance we created as seen below:

Verifying the data:

Then the following datasets were loaded into Neo4j using the codes and these were the following outputs:

- Airline dataset:

```
//Checking to see if the airline data loaded properly.  
LOAD CSV WITH HEADERS FROM 'file:///airline.csv' AS row  
RETURN row  
LIMIT 1;
```

- Which gives the following output for the airline dataset:

```
{  
  airline_name:"Tuna Aero",  
  airline_country:"Sweden"  
}
```

- Airport dataset:

```
//Checking to see if the airport data loaded properly.  
LOAD CSV WITH HEADERS FROM 'file:///airport.csv' AS row  
RETURN row  
LIMIT 1;
```

- Which gives the following output for the airport dataset:

```
{  
  airport_name:"Norsup Airport",  
  airport_city:"Norsup",  
  airport_country_region:"Vanuatu"  
}
```

- Serves_from dataset:

```
//Checking to see if the serves_from relationship loaded properly.  
LOAD CSV WITH HEADERS FROM 'file:///serves_from.csv' AS row  
RETURN row  
LIMIT 1;
```

- Which gives the following output for the serves_from dataset:

```
{
  airline_name:"Tuna Aero",
  airport_name:"Norsup Airport",
  airport_city:"Norsup",
  airport_country_region:"Vanuatu"
}
```

- Route dataset:

```
//Checking to see if the route relationship loaded properly.
LOAD CSV WITH HEADERS FROM 'file:///route.csv' AS row
RETURN row
LIMIT 1
```

```
{
  departure_airport_country_region:"Vanuatu",
  departure_airport_city:"Norsup",
  arrival_airport_name:"Bauerfield International Airport",
  arrival_airport_country_region:"Vanuatu",
  plane_type:"Harbin Yunshuji Y12;De Havilland Canada DHC-6 Twin Otter;Pilatus Britten-Norman BN-2A/B Islander",
  departure_airport_name:"Norsup Airport",
  arrival_airport_city:"Port-vila",
  airline_name:"Tuna Aero"
}
```

From the code chunks and the outputs above, we could confirm that the datasets have been loaded successfully into Neo4j for verification. Now we could proceed with creating nodes and relationships among them.

Creating Nodes and Relationships

The next step was to generate the graph nodes and relationships after confirming that all CSV files had been correctly loaded into Neo4j. Each CSV file was read from the Neo4j import folder using the LOAD CSV WITH HEADERS command. The nodes and the SERVES_FROM connection were created using the MERGE clause, which determines

whether a matching record already exists before generating a new one. After matching the airline using `airline_name`, the `airline_country` property was assigned for the Airline node using the SET clause. Prior to establishing connections between them, the MATCH clause was utilized to locate already existing airlines and airport nodes. However, because each row in the dataset represents a distinct route record, and duplicate route records need to be retained for count-based queries. Hence, CREATE was used for the ROUTE relationship. Lastly, `plane_type` property was created by splitting the `plane_type` field using the semicolon (;) separator, because some route records contain more than one aircraft type and stored as a list. The nodes and the relationships were loaded using the following codes:

Code for airline node:

```
LOAD CSV WITH HEADERS FROM 'file:///airline.csv' AS row
WITH row
WHERE row.airline_name IS NOT NULL AND row.airline_name <> ""
MERGE (a:Airline {
  airline_name: row.airline_name
})
SET a.airline_country = row.airline_country;
```

Code for airport node:

```
LOAD CSV WITH HEADERS FROM 'file:///airport.csv' AS row
WITH row
WHERE row.airport_name IS NOT NULL AND row.airport_name <> ""
MERGE (a:Airport {
  airport_name: row.airport_name,
  airport_city: row.airport_city,
  airport_country_region: row.airport_country_region
});
```

Code for SERVES_FROM relationship:

```
LOAD CSV WITH HEADERS FROM 'file:///serves_from.csv' AS row
MATCH (airline:Airline {
  airline_name: row.airline_name
})
MATCH (airport:Airport {
```

```
airport_name: row.airport_name,  
airport_city: row.airport_city,  
airport_country_region: row.airport_country_region  
})  
MERGE (airline)-[:SERVES_FROM]->(airport);
```

Code for ROUTE relationship:

```
LOAD CSV WITH HEADERS FROM 'file:///route.csv' AS row  
MATCH (first_airport:Airport {  
  airport_name: row.departure_airport_name,  
  airport_city: row.departure_airport_city,  
  airport_country_region: row.departure_airport_country_region  
})  
MATCH (second_airport:Airport {  
  airport_name: row.arrival_airport_name,  
  airport_city: row.arrival_airport_city,  
  airport_country_region: row.arrival_airport_country_region  
})  
CREATE (first_airport)-[:ROUTE {  
  airline_name: row.airline_name,  
  plane_type: [plane IN split(row.plane_type, ',') | trim(plane)]  
}]->(second_airport);
```

Cypher Queries

Query - 1

This query finds all Airline nodes where the airline_country property is Australia. The MATCH clause identifies airline nodes, and the WHERE clause filters them by country which is Australia, and then the RETURN DISTINCT clause outputs the unique airline names only.

```
MATCH (airline:Airline)  
WHERE airline.airline_country = "Australia"  
RETURN DISTINCT airline.airline_name AS australia_airline_names;
```

Output:

```
australia_airline_names
1  "Whyalla Airlines"
2  "Transpac Express"
3  "Transaustralian Air Express"
```

Query – 2

This query classifies each route as either domestic or international by comparing the airport_country_region of the departure and the arrival airports. If both airports are in the same country/region, the route is classified as a domestic airport, otherwise, it is classified as an international airport. The count(route) function then counts the number of route records in each category.

```
MATCH (departure_airport:Airport)-[route:ROUTE]->(arrival_airport:Airport)
RETURN
CASE
WHEN departure_airport.airport_country_region = arrival_airport.airport_country_region
THEN "Domestic"
ELSE "International"
END AS route_type,
count(route) AS route_count;
```

Output:

route_type	route_count
"Domestic"	26411
"International"	30890

Query – 3

This query identifies the airport pair with the greatest number of route records. The MATCH clause finds all ROUTE relationships between airports. The CASE statements arrange the two airport names in a consistent alphabetical order so that routes in opposite directions, such as $A \rightarrow B$ and $B \rightarrow A$, are treated as the same airport pair. The query then counts the number of ROUTE relationships for each airport pair and returns the pair with the highest count.

```
MATCH (first_airport:Airport)-[route:ROUTE]->(second_airport:Airport)
WITH
CASE
WHEN first_airport.airport_name < second_airport.airport_name
THEN first_airport.airport_name
ELSE second_airport.airport_name
END AS airport_1,
CASE
WHEN first_airport.airport_name < second_airport.airport_name
THEN second_airport.airport_name
ELSE first_airport.airport_name
END AS airport_2,
count(route) AS route_count
RETURN
airport_1,
airport_2,
route_count
ORDER BY route_count DESC
LIMIT 1;
```

Output:

airport_1	airport_2	route_count
"Chicago O'Hare International Airport"	"Hartsfield Jackson Atlanta International Airport"	37

Query – 4

This query identifies the top five airport pairs with the greatest number of distinct plane types used across their route records. Since plane_type is stored as a list, UNWIND is used to separate each plane type as an individual value. The CASE statements arrange the two airport names in an order so that $A \rightarrow B$ and $B \rightarrow A$ are treated as the same airport pair. The query then counts the distinct plane types for each airport pair and returns the five pairs with the highest counts.

```
MATCH (first_airport:Airport)-[route:ROUTE]->(second_airport:Airport)
UNWIND route.plane_type AS plane
WITH
CASE
WHEN first_airport.airport_name < second_airport.airport_name
THEN first_airport.airport_name
ELSE second_airport.airport_name
END AS airport_1,
CASE
WHEN first_airport.airport_name < second_airport.airport_name
THEN second_airport.airport_name
ELSE first_airport.airport_name
END AS airport_2,
plane
RETURN
airport_1,
airport_2,
count(DISTINCT plane) AS distinct_plane_types
ORDER BY distinct_plane_types DESC
LIMIT 5;
```

Output:

airport_1	airport_2	distinct_plane_types
"Hong Kong International Airport"	"Incheon International Airport"	15
"Hong Kong International Airport"	"Suvarnabhumi Airport"	13
"Beijing Capital International Airport"	"Guangzhou Baiyun International Airport"	13
"Incheon International Airport"	"Kansai International Airport"	12
"Beijing Capital International Airport"	"Shanghai Hongqiao International Airport"	12

Query – 5

This query finds the total distinct travel routes from Beijing Capital International Airport to Perth International Airport using at most three ROUTE relationships. The variable-length relationship pattern `[ :ROUTE*1..3 ]` allows one, two, or up to three route connections between the two airports. The `nodes(path)` function extracts the sequence of airports visited in each path, and `DISTINCT` ensures that duplicate airport sequence is counted only once. Finally, `count(airport_sequence)` returns the total number of distinct route sequences.

```
MATCH path =
(:Airport{airport_name: "Beijing Capital International Airport"})
-[:ROUTE*1..3]->
(:Airport{airport_name: "Perth International Airport"})
WITH DISTINCT [airport IN nodes(path) | airport.airport_name] AS airport_sequence
RETURN count(airport_sequence) AS distinct_route_count;
```

Output:

distinct_route_count
650

Query – 6

This query identifies the top five pairs of competing airlines based on the number of airport pairs they both operate on. The MATCH clause finds all route relationships between airports. CASE arranges each airport pair in a consistent order so that routes A → B and B → A are treated as the same airport pair. For each airport pair, the query collects the distinct airlines operating on that pair. Airport pairs with only one airline are removed as there is no competition among them. The UNWIND clauses generate airline pair combinations, and the WHERE clause avoids duplicate pairs and self pairing. Finally, the query counts how many airport pairs the airlines share, and returns the top five competing airline pairs.

```
MATCH (first_airport:Airport)-[route:ROUTE]->(second_airport:Airport)
WITH
CASE
WHEN first_airport.airport_name < second_airport.airport_name
THEN first_airport.airport_name
ELSE second_airport.airport_name
END AS airport_1,
CASE
WHEN first_airport.airport_name < second_airport.airport_name
THEN second_airport.airport_name
ELSE first_airport.airport_name
END AS airport_2,
collect(DISTINCT route.airline_name) AS airlines
WHERE size(airlines) > 1
UNWIND airlines AS airline_1
UNWIND airlines AS airline_2
WITH airport_1, airport_2, airline_1, airline_2
WHERE airline_1 < airline_2
RETURN
```

```

airline_1,
airline_2,
count(*) AS shared_airline_routes
LIMIT 5;

```

Output:

airline_1	airline_2	shared_airline_routes
"SAM Colombia"	"Zantop International Airlines"	795
"AtlasGlobal Ukraine"	"Sheremetyevo-Cargo"	359
"Servicos De Alquile r"	"Sheremetyevo-Cargo"	244
"Sham Wing Airlines"	"Thai Flying Helicopte r Service"	208
"Sham Wing Airlines"	"Star Aviation"	202

Self-designed queries

Query – 1 Cypher Query:

Display the top 5 busiest airports details along with the number of routes in the airports.

This query identifies the top five airports with the highest number of routes recorded. The MATCH clause finds all ROUTE relationships starting from each departure airport. The count(route) function then counts how many route records depart from each airport. The results are ordered in descending order so that the busiest departure airports appear first.

```

MATCH (departure_airport:Airport)-[route:ROUTE]->(arrival_airport:Airport)
RETURN
departure_airport.airport_name AS departure_airport,

```

```
departure_airport.airport_city AS city,
departure_airport.airport_country_region AS country_region,
count(route) AS number_of_departing_routes
ORDER BY number_of_departing_routes DESC
LIMIT 5;
```

Output:

departure_airport	city	country_region	number_of_routes
"Hartsfield Jacks on Atlanta Intern ational Airport"	"Atlanta"	"United States"	759
"Beijing Capital International Air port"	"Beijing"	"China"	519
"London Heathrow Airport"	"London"	"United Kingdom"	516
"Charles de Gaull e International A irport"	"Paris"	"France"	507
"Chicago O'Hare I nternational Airp ort"	"Chicago"	"United States"	485

Query – 2 APOC Query:

Which airport pair has the most competition among airlines?

This APOC query identifies the airport pair with the highest number of distinct airlines operating amongst themselves. Unique airline names are collected for each airport pair and only pairs that serve more than one airline are kept. Size(airlines) counts the number of airlines, while apoc.text.join() converts the list of airline names into a more readable format which would be ‘,’ separated. The results are ordered by the number of airlines in

descending order, and LIMIT 1 returns the most competitive airport pair in the graph database.

```
MATCH (departure_airport:Airport)-[route:ROUTE]->(arrival_airport:Airport)
WITH
departure_airport,
arrival_airport,
collect(DISTINCT route.airline_name) AS airlines
WHERE size(airlines) > 1
RETURN
departure_airport.airport_name AS departure_airport,
arrival_airport.airport_name AS arrival_airport,
size(airlines) AS number_of_airlines,
apoc.text.join(airlines, ", ") AS airlines_operating
ORDER BY number_of_airlines DESC
LIMIT 1;
```

departure_airport	arrival_airport	number_of_airlines	airlines_operating
"Chicago O'Hare International Airport"	"Hartsfield Jackson Atlanta International Airport"	19	"Sunbird Airlines, Skydrift, SAM Colombia, Ues-Avia Aircompany, Zantop International Airlines, Slate Falls Airways, TAG Aviation USA, Top Sky International, Surninam International Victory Airline, Eagle Canyon Airlines, XP International, Cathay Pacific, Sunshine Air Tours, Trabajos Aereos Vascongados, Universal Jet, Star Aviation, Thai Flying Helicopter Service, Wind Rose Aviation, Sham Wing Airlines"

4. Graph Data Science

Introduction

Graph Data Science is the use of graph structures and graph algorithms to analyse relationships between connected entities. Unlike analysis of tables, which mainly focuses on individual records, graph data science focuses on how entities are connected and how those connections influence flow through a network. Neo4j Graph Data Science includes algorithm categories such as similarity, path finding, etc. which can be applied to different real-world problems [5].

Practical Application of Graph Data Science

Navigation systems and other route-finding applications draw on graph data science on an algorithm namely path finding [6]. In this scenario, roads can be represented as relationships with weights like trip time or distance, while locations can be represented as nodes. Then, potential routes between places are found using path-finding algorithms, and the most effective one is determined using the chosen weight.

Dijkstra's algorithm [7], which determines the shortest or least expensive path between two places in a weighted graph, is a good fit for this application. For example, depending on whether the relationship weight is based on trip time or distance, the algorithm can determine the quickest or shortest path when traveling from The University of Western Australia to Yagan Square. This makes path-finding useful for applications such as navigation, delivery planning, transport optimisation, travel route recommendations, path a robot follows, delivery planning, game path finding (pac man) etc.

The pros of using Graph Data Science for route-finding is that transport networks naturally fit a graph structure, where locations are nodes and roads or routes are relationships. Algorithms such as Dijkstra's shortest path can find the most efficient route based on weights such as distance, travel time, or cost.

However, the cons of the algorithm is such that the accuracy of the result depends on the quality of the relationship weights. If travel time, distance, available routes or traffic data is outdated, the suggested route may not be optimal. Real world applications could also be more complex because conditions such as congestion, road closures, and weather could change frequently.

5. Metadata

Metadata is important as in a graph database it describes the structure of the graph database, which would include nodes, labels, relationship, and properties [8]. In Neo4j, labels are used to classify nodes. And relationships describe how the nodes are connected amongst each other. And properties are used to store values on nodes and relationships. This would be useful during the design stage as it helps identify what should be designed as nodes, relationship, and what attributes should be stored in the properties [9].

In this project, thinking about metadata helped with designing the Arrows App design. The main node were Airline and Airport, because these represent the key entities in the airline route dataset. The main relationship types identified were SERVES_FROM and ROUTE,

because they describe how airlines and airports are connected. Properties such as `airline_name`, `airline_country`, `airport_name`, `airport_city`, `airport_country_region`, and `plane_type` were then assigned to the relevant nodes and relationships.

Metadata is also used while the development of cypher query because the queries depend on using the correct labels (nodes), relationship, and properties. For example, if the graph database using the incorrect label, or relationship would result with errors in the output.

After the data was loaded into Neo4j, metadata could be inspected and validated. The following queries were used to check the graph metadata, along with their output:

- **Label (nodes)**

Query:

```
CALL db.labels();
```

Output:

```
label
"Airline"
"Airport"
```

- **Nodes count**

- Query:

```
MATCH (n)
RETURN labels(n) AS node_type, count(n) AS nodes_count;
```

- Output:

node_type	nodes_count
["Airline"]	488
["Airport"]	2800

The output confirms that the number of nodes in the graph database is equal to that of the csv file loaded into Neo4j. It could also be noted that the counts match for both the nodes.

- **Relationships**

Query:

```
CALL db.relationshipTypes();
```

Output:

relationshipType
"SERVES_FROM"
"ROUTE"

- **Relationships count**

- Query:

```
MATCH ()-[r]->()
RETURN type(r) AS relationship_type, count(r) AS relationships_count;
```

Output:

relationship_type	relationship_count
"SERVES_FROM"	16768
"ROUTE"	57301

The routes relationship count which is 57301 matches to that of the dataset Project2_dataset.csv, and this proves that all the route information was mapped correctly. The same could be noted for the serves_from relationship as the count matches the rows in the csv file imported.

- **Properties**

Query:

```
CALL db.propertyKeys();
```

Output:

propertyKey
"airline_name"
"airline_country"
"airport_city"
"airport_country_region"
"airport_name"
"plane_type"

After the data was loaded into Neo4j, the metadata was inspected using:

- The CALL db.labels() query was used to confirm the node labels in the database.
- CALL db.relationshipTypes() was used to confirm the relationships in place.
- And CALL db.propertyKeys() was used to identify the properties available while a developer or data engineer writes queries.

These queries confirm that the implemented graph database matches the arrows app design and help in ensuring that the correct labels, relationships, and properties were used while writing cypher queries.

6. References:

[1] "Graph database concepts - Getting Started," Neo4j Graph Data Platform.

<https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/>

[2] "What is a graph database - Getting Started," Neo4j Graph Data Platform, 2024.

<https://neo4j.com/docs/getting-started/graph-database/>

[3] T. Searle, "What is the hub and spoke model?," TransVirtual, Sep. 04, 2022.

<https://www.transvirtual.com/blog/what-is-the-hub-and-spoke-model/>

[4] "Wikipedia," Wikipedia.org, Jan. 15, 2001. <https://www.wikipedia.org/>

[5] "Introduction - Neo4j Graph Data Science," Neo4j Graph Data Platform.

<https://neo4j.com/docs/graph-data-science/current/introduction/>

[6] "What is Graph Data Science? Essential Overview (6 Minute Article)," Graphable, Feb.

17, 2023. <https://graphable.ai/blog/graph-data-science/>

[7] "Dijkstra's algorithm," Wikipedia, May 28, 2019.

https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm

[8] "List functions - Cypher Manual," Neo4j Graph Data Platform.

<https://neo4j.com/docs/cypher-manual/current/functions/list/>

[9] "Graph database concepts - Getting Started," Neo4j Graph Data Platform.

<https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/>